Module 05

Vector Embeddings & RAG

How machines represent meaning — and how to build semantic search

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. The Problem: Keyword Search is Dumb

If you search a database for 'car', you'll miss articles about 'automobile', 'vehicle', or 'Tesla'. Keyword search matches exact strings. **Semantic search** understands meaning — 'car' and 'automobile' are similar. This is made possible by **vector embeddings**.

2. What is a Vector Embedding?

An embedding is a list of numbers (a vector) that represents the meaning of a piece of text. An embedding model (a neural network) converts any sentence into, say, 768 numbers. Sentences with similar meanings produce vectors that are close to each other in 768-dimensional space.

```
# Conceptual example (actual vectors are 768 numbers)
"The stock market crashed"   → [0.23, -0.41, 0.88, ...]
"Markets fell sharply today" → [0.25, -0.39, 0.86, ...]   # very similar!
"I like pizza"               → [-0.91, 0.12, -0.33, ...]  # very different
```

3. Cosine Similarity — How Close are Two Vectors?

Cosine similarity measures the angle between two vectors. Score = 1.0 means identical meaning, 0.0 means unrelated, -1.0 means opposite.

```
import numpy as np

def cosine_similarity(vec_a: list[float], vec_b: list[float]) -> float:
    a = np.array(vec_a)
    b = np.array(vec_b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# Example:
v1 = [1.0, 0.0, 0.0]
v2 = [0.9, 0.1, 0.0]
v3 = [0.0, 1.0, 0.0]

print(cosine_similarity(v1, v2))  # ~0.99 — very similar
print(cosine_similarity(v1, v3))  # 0.0   — completely different
```

4. Generating Embeddings with an API

```
# Option A: Google Gemini text-embedding-004 (API-based, 768 dims)
from google import genai

client = genai.Client(api_key="YOUR_GOOGLE_API_KEY")
result = client.models.embed_content(
    model="models/text-embedding-004",
    contents=["Stock market crashes 10%"],
    config={"task_type": "RETRIEVAL_DOCUMENT"},
)
vector = result.embeddings[0].values   # list of 768 floats

# Option B: Local fastembed BAAI/bge-base-en-v1.5 (no API key, 768 dims)
from fastembed import TextEmbedding
model = TextEmbedding(model_name="BAAI/bge-base-en-v1.5")
```

```
vectors = list(model.embed(["Stock market crashes 10%"]))
vector = vectors[0].tolist()
```

5. pgvector — Vector Search in PostgreSQL

pgvector is a Postgres extension that adds a vector column type and approximate nearest-neighbor search. This means you can store embeddings in Postgres and query for the most similar ones — no separate vector database needed.

```
-- 1. Enable the extension (run once in Supabase SQL editor)
CREATE EXTENSION IF NOT EXISTS vector;

-- 2. Create a table with a vector column
CREATE TABLE knowledge_base (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_file TEXT,
    chunk_index INT,
    content     TEXT,
    embedding   vector(768),   -- 768-dimensional vector
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Index for fast approximate search (HNSW algorithm)
CREATE INDEX ON knowledge_base
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

6. Storing and Querying Embeddings from Python

```
# Storing an embedding
supabase.table("knowledge_base").insert({
    "source_file": "rbi_policy.pdf",
    "chunk_index": 0,
    "content": "RBI raised rates by 25 bps...",
    "embedding": [0.23, -0.41, 0.88, ...],  # 768 floats
}).execute()

# Searching: find 5 most similar chunks to a query
# (uses a Postgres function defined in Supabase)
resp = supabase.rpc(
    "match_knowledge_base",
    {"query_embedding": query_vector, "match_count": 5}
).execute()
results = resp.data
```

```
-- The match_knowledge_base function in Postgres:
CREATE OR REPLACE FUNCTION match_knowledge_base(
    query_embedding vector(768),
    match_count     int
)
RETURNS TABLE (id uuid, content text, similarity float)
LANGUAGE SQL
AS $$
    SELECT id, content,
```

```
        1 - (embedding <=> query_embedding) AS similarity
    FROM   knowledge_base
    ORDER  BY embedding <=> query_embedding
    LIMIT  match_count;
$$;
```

7. RAG — Retrieval-Augmented Generation

RAG combines semantic search with an LLM. Instead of asking the LLM to answer from its training data alone (which may be outdated or wrong), you first retrieve relevant documents from your database, then put them in the LLM's context.

```
# Full RAG pipeline:
def answer_with_rag(question: str) -> str:
    # Step 1: embed the question
    query_vector = embed_text(question)

    # Step 2: retrieve top-5 relevant KB chunks
    resp = supabase.rpc(
        "match_knowledge_base",
        {"query_embedding": query_vector, "match_count": 5}
    ).execute()
    context = "\n\n".join(r["content"] for r in resp.data)

    # Step 3: ask the LLM using the retrieved context
    response = anthropic.messages.create(
        model="claude-sonnet-4-6",
        system="Answer only based on the context provided.",
        messages=[{
            "role": "user",
            "content": f"Context:\n{context}\n\nQuestion: {question}",
        }],
        max_tokens=1024,
    )
    return response.content[0].text
```

8. Text Chunking — Splitting Documents for Embedding

LLM context windows and embedding models have token limits. Long documents must be split into chunks. The strategy matters — split too small and you lose context, too large and you exceed limits.

```
def chunk_text(text: str, chunk_size: int = 800, overlap: int = 100) -> list[str]:
    """Split text into overlapping chunks of ~chunk_size words."""
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        chunks.append(chunk)
        start += chunk_size - overlap   # overlap helps with continuity
    return chunks
# Embed all chunks and store in DB
```

```
text = "Very long document..."
chunks = chunk_text(text)
for i, chunk in enumerate(chunks):
    vector = embed_text(chunk)
    supabase.table("knowledge_base").insert({
        "source_file": "doc.pdf",
        "chunk_index": i,
        "content": chunk,
        "embedding": vector,
    }).execute()
```

9. The FallbackEmbedder Pattern

ContentAutomation2 uses a fallback pattern: try Gemini first, fall back to local fastembed if Gemini fails. This makes the system resilient — it works even without API access. See backend/app/agents/embedding/client.py.

10. Practice Exercises

- Exercise 1: Install fastembed: pip install fastembed. Embed 5 sentences and print the first 5 values of each vector.
- Exercise 2: Compute cosine similarity between 'stock market crash' and 'equity markets fell'.
- Exercise 3: Enable pgvector in Supabase. Create a kb_demo table with a vector(768) column.
- Exercise 4: Embed 10 sentences and insert them into kb_demo with their vectors.
- Exercise 5: Create a match_kb_demo Postgres function. Query for the 3 most similar to 'interest rates rise'.
- Exercise 6: Build the full RAG pipeline: question → embed → retrieve → LLM answer.